Лабораторна робота № 2

Оптимізація багатоекстремальної функції в дискретному просторі з обмеженнями з використанням алгоритму імітації відпалу

Виконав: Олійник Гліб, група ІПЗ-24-2, Варіант № 25

Мета роботи: вивчення стохастичних методів оптимізації, практичне освоєння алгоритму імітації відпалу та дослідження його ефективності під час пошуку прихованих глобальних екстремумів в умовах статичних просторових обмежень.

Завдання до лабораторної роботи:

1. Реалізувати клієнтську програму, що підключається до сервера (ooi-lab2.up.railway.app) за наданим API.

2. Підібрати гіперпараметри алгоритму імітації відпалу (початкова температура Tstart, закон охолодження, функція залежності радіуса R(T)).

3. Домогтися успішного знаходження всіх прихованих об'єктів на карті свого варіанта.

4. Реалізувати візуалізацію руху агента в реальному часі. На карті мають відображатися перешкоди, робот і постійні позначки місць, де об'єкти було успішно знайдено.

Хід роботи

1. Короткий теоретичний опис алгоритму імітації відпалу та критерію Метрополіса

Імітація відпалу - стохастичний метод глобальної оптимізації, що повторює охолодження металу. Поки температура T висока, агент вільно приймає і гірші кроки та виходить із локальних максимумів, а коли T спадає, пошук стає точнішим.

Агент рухається сіткою 100 x 100, а сигнал S(x, y) від сервера потрібно максимізувати, тому енергія E = -S.

Перехід у нову клітинку приймається за критерієм Метрополіса. Зміна енергії:

$$\Delta E = Enew - Eold, \text{ де } E = -S$$

Якщо $\Delta E <= 0$, крок приймається завжди. Якщо $\Delta E > 0$, то з імовірністю:

$$P = \exp(-\Delta E / T)$$

Випадкове число ξ з рівномірного розподілу порівнюється з P: при $\xi < P$ агент переходить у гіршу клітинку, інакше лишається на місці. Чим вища T, тим частіше приймаються погіршення.

Температура знижується за одним із законів охолодження (k - номер ітерації, α - коефіцієнт):

геометричний: $T_{k+1} = T_k * \alpha$, де $0,8 <= \alpha <= 0,99$

лінійний: $T_{k+1} = T_0 - k * \Delta t$

логарифмічний: $T_k = T_0 / \ln(1 + k)$

Радіус пошуку нової клітинки залежить від температури:

$$R(T) = round(Rmax * T / Tstart)$$

Після знаходження об’єкта його пік зникає, тому застосовується повторне нагрівання: температуру скидають до Tstart, і радіус R знову стає великим.

2. Алгоритм пошуку

Клієнт написано на Python зі стандартним urllib. Функція start_session() викликає POST /api/start і отримує session_id, стартову позицію та карту перешкод, а send_move() викликає POST /api/move і повертає signal_level, fox_caught, remaining_foxes та steps_made. Якщо запит не вдався через мережу, він повторюється до 5 разів, а відповідь 400 одразу передається далі.

Лістинг коду:

```
def post_json(path, payload):
    data = json.dumps(payload).encode("utf-8")
    for attempt in range(5):
        request = urllib.request.Request(
            BASE_URL + path, data=data,
```

```
                headers={"Content-Type": "application/json"})
            try:
                with urllib.request.urlopen(request, timeout=30) as response:
                    return json.loads(response.read().decode("utf-8"))
            except urllib.error.HTTPError:
                raise
            except Exception:
                if attempt == 4:
                    raise


def send_move(session_id, x, y):
    return post_json("/api/move", {
        "session_id": session_id,
        "target_position": {"x": int(x), "y": int(y)}})
```

Запит у клітинку з перешкодою рахується як крок і повертає помилку, тому координати перевіряються локально. Нового кандидата агент вибирає лише серед вільних клітинок карти (метод free()), у крузі радіуса R навколо себе. Клітинки, які вже опитувалися, зберігаються в кеші, і за можливості вибирається нова клітинка, щоб не витрачати кроки на повторні запити.

Гіперпараметри, з якими було виконано проходження:

Параметр	Значення	Призначення
Tstart	25	початкова температура
Закон охолодження	геометричний	Tk+1 = Tk * α
α	0,9	коефіцієнт охолодження
Rmax	45	найбільший радіус пошуку, R(T) = round(Rmax * T / Tstart)
stall	40	кількість запитів без покращення, після якої температура скидається
Tmin	0,05	нижня межа температури (R не менший за 1)

Нижче наведено псевдокод алгоритму разом із повторним нагріванням.

Псевдокод:

```
Початок:
    T = T_start
    S = сигнал у стартовій клітинці
    найкращий сигнал = S, лічильник застою = 0

Повторювати, поки є не знайдені об’єкти:
    1. Радіус пошуку: R = round(R_max * T / T_start), не менше 1.
    2. Вибрати випадкову вільну клітинку (x1, y1) на відстані
       не більше R від агента (перевага - ще не опитаним).
    3. Отримати сигнал S1 у цій клітинці (запит до сервера або кеш).
    4. Різниця енергій: dE = S - S1.
    5. Імовірність: P = 1, якщо dE <= 0, інакше P = exp(-dE / T).
    6. Якщо випадкове число менше P, агент переходить у (x1, y1),
       і S = S1.
    7. Якщо S1 більший за найкращий сигнал, запам’ятати його
       і обнулити лічильник застою, інакше збільшити лічильник на 1.
    8. Якщо сервер повідомив, що об’єкт знайдено:
         - зберегти координати, очистити кеш і найкращий сигнал;
         - якщо об’єктів більше немає, завершити пошук;
         - повторне нагрівання: T = T_start;
         - заново отримати сигнал S у поточній клітинці;
         - повернутися до кроку 1.
    9. Охолодження: T = T * alpha (не нижче T_min).
    10. Якщо лічильник застою досяг stall, агент повертається
        в найкращу відому клітинку і T = T_start.
```

Охолодження, радіус і критерій Метрополіса реалізовано окремими функціями:

Лістинг коду:

```
def cooling(law, t_start, t_current, k, alpha, delta_t):
    if law == LINEAR:
        return t_start - k * delta_t
    if law == LOGARITHMIC:
        return t_start / math.log(1.0 + k)
    return t_current * alpha


def radius(t_current, t_start, r_max):
    value = int(round(r_max * t_current / t_start))
    if value < 1:
```

```
            return 1
        if value > r_max:
            return r_max
        return value


    def acceptance(delta_e, t_current):
        if delta_e <= 0.0:
            return 1.0
        if t_current <= 0.0:
            return 0.0
        power = -delta_e / t_current
        if power < -700.0:
            return 0.0
        return math.exp(power)
```

Після знаходження об’єкта викликається catch(): кеш і найкращий сигнал очищаються, бо рельєф змінився, а reheat() скидає температуру до Tstart. Те саме робить jump_to_best() із reheat() при застої. Для останнього об’єкта повторне нагрівання не потрібне, і пошук завершується.

Лістинг коду:

```
def reheat(self):
    self.temperature = self.params.t_start
    self.iteration = 0
    self.stall = 0
    self.reheats += 1
    self.temperature_log.append((self.requests, self.temperature))


def catch(self, x, y):
    self.found.append((x, y))
    self.cache = {}
    self.best_signal = -1.0
    self.best_position = (x, y)
    self.signal = None
    if self.remaining == 0:
        self.finished = True
    else:
        self.reheat()
```

3. Графік зміни температури

Графік будується в правій частині вікна програми (панель “Температура T(k)”). По осі X відкладено номер запиту, по осі Y температуру. Червоні штрихові лінії позначають моменти повторного нагрівання (Рис. 1).

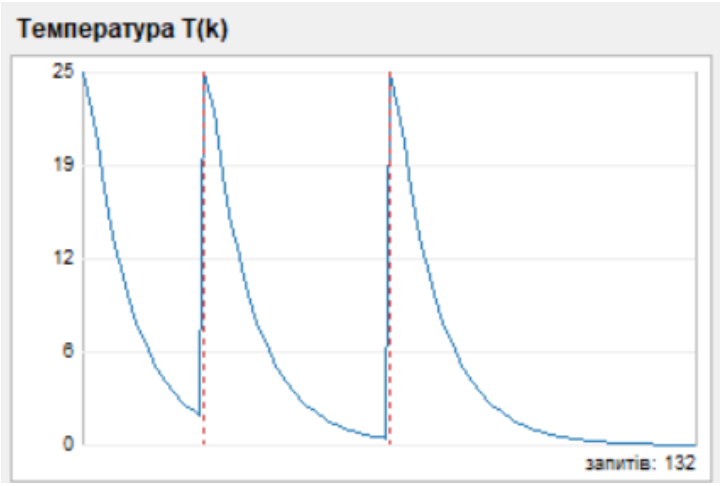


Рис. 1. Зміна температури T від номера запиту

На графіку три цикли охолодження. Перший завершується знаходженням першого об’єкта (запит 26), потім температура стрибком повертається до 25 (перше нагрівання). Другий цикл триває до запиту 66 (друге нагрівання), а третій - до знаходження останнього об’єкта. Протягом кожного циклу температура спадає за геометричним законом приблизно до 0 - від широкого пошуку до точного підходу.

4. Фінальне вікно візуалізації

Знімок вікна програми після завершення пошуку наведено на Рис. 2. На карті зеленим показано перешкоди, світло-блакитним - клітинки, в які агент перейшов, світло-сірим - опитані клітинки, куди перехід не відбувся. Червоний квадрат - агент (його останнє положення), а оранжеві позначки з рамкою - місця, де об’єкти знайдено. Вони залишаються на карті постійно. Праворуч розташовані графік температури, показники пошуку та список знайдених об’єктів, а над картою - панель гіперпараметрів і кнопки керування.

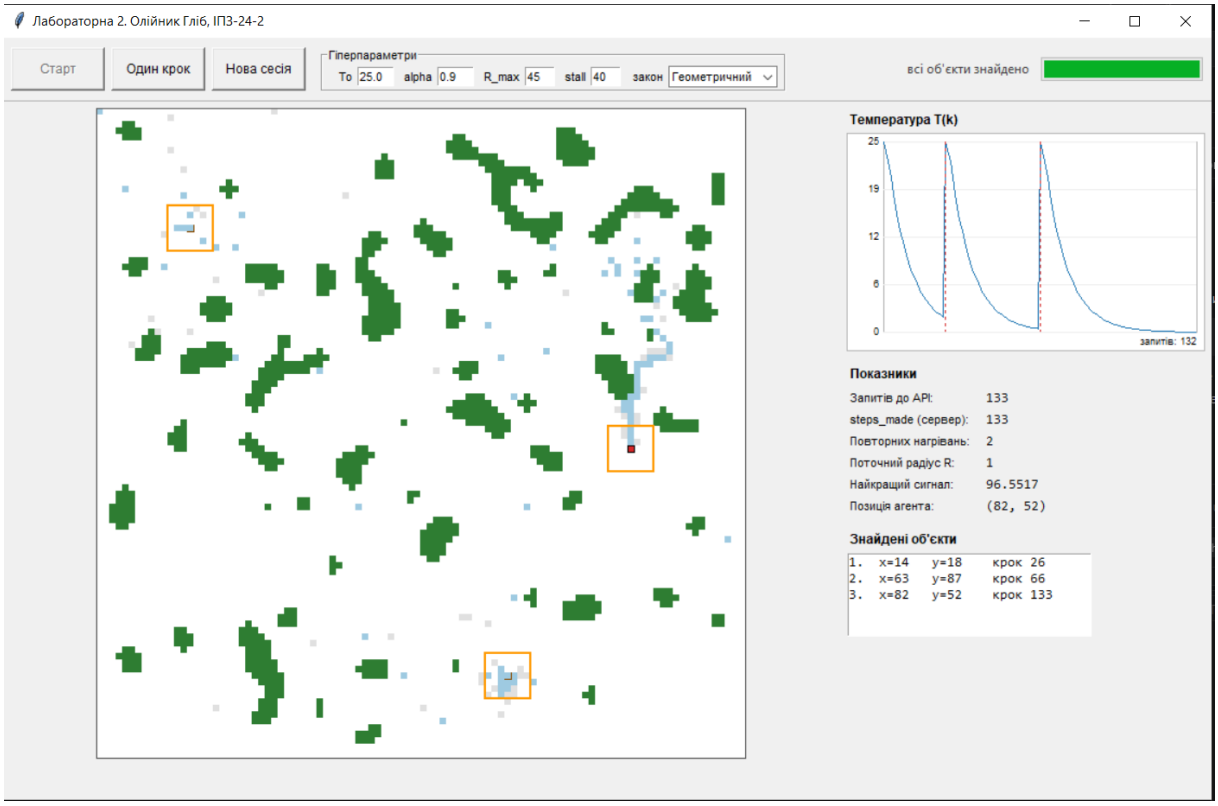


Рис. 2. Фінальне вікно візуалізації після знаходження всіх об'єктів

5. Підсумкова кількість кроків

Усі три приховані об’єкти знайдено. Лічильник кроків на сервері (steps_made) дорівнює кількості запитів клієнта, тобто 133. Запитів у перешкоди не було, оскільки координати перевіряються за картою. З 133 запитів 2 витрачено на повторне вимірювання сигналу в поточній клітинці після знаходження перших двох об’єктів, бо рельєф після цього змінюється. Було виконано 2 повторні нагрівання. Порядок знаходження об’єктів показано в таблиці.

№	x	y	Крок знаходження	Кроків на об’єкт
1	14	18	26	26
2	63	87	66	40
3	82	52	133	67
Разом			133	133

Найбільше значення сигналу під час пошуку - 96,5517.

Висновки: у ході роботи реалізовано клієнт для сервера та алгоритм імітації відпалу з критерієм Метрополіса, геометричним охолодженням, радіусом R(T) і повторним нагріванням. Для варіанта 25 (5) (Tstart = 25, α = 0,9, Rmax = 45) знайдено всі три приховані об’єкти за 133 кроки (26, 40 і 67 кроків на об’єкт), запитів у перешкоди не було.

Коефіцієнт α задає швидкість охолодження. Чим він менший, тим швидше радіус R звужується до 1 (через 16 ітерацій при α = 0,8, 33 при α = 0,9 і 339 при α = 0,99): пошук швидший, але зростає ризик застрягти в хибному максимумі. Велика α дає надійніший пошук, але потребує більше кроків. Стартова температура Tstart впливає лише на ймовірність прийняття гірших кроків, а швидкість звуження радіуса від неї не залежить.

Повторне нагрівання необхідне: без нього охолоджений агент після зникнення піку не знайшов би наступний об’єкт.